# Tarea 2: Conexión de Octave con Postgre

Maria Fernanda Alvarez Morales, 200201537[1, *]

[1] *Facultad de Ingeniería, Escuela de Mecánica Eléctrica, Universidad de San Carlos de Guatemala*

En este trabajo se documenta el proceso de conexión entre el software de cálculo numérico Octave y la base de datos PostgreSQL. Se detalla cómo instalar los componentes necesarios, la creación de tablas SQL y la ejecución de consultas desde Octave usando el paquete `database`. Este procedimiento permite la integración eficiente entre bases de datos y entornos de programación científica.

## I. INTRODUCCIÓN

GNU Octave es un software de código abierto diseñado para realizar cálculos numéricos. PostgreSQL, por su parte, es un sistema gestor de bases de datos relacional muy robusto. Integrar ambas herramientas permite automatizar análisis de datos y crear aplicaciones que involucren almacenamiento y recuperación estructurada de información. Esta práctica tiene como objetivo demostrar cómo se pueden insertar y consultar datos desde Octave usando una base de datos PostgreSQL.

## II. MARCO TEÓRICO

### Bases de Datos Relacionales y PostgreSQL

Una base de datos relacional es un sistema que almacena información en tablas (relaciones) que pueden estar interconectadas por claves primarias y foráneas. Estas bases de datos utilizan SQL (Structured Query Language) como lenguaje estándar para definir, manipular y consultar los datos. PostgreSQL es un sistema de gestión de bases de datos relacional (RDBMS) de código abierto, reconocido por su robustez, escalabilidad y cumplimiento de estándares ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).

PostgreSQL permite trabajar con tipos de datos complejos, funciones definidas por el usuario y operaciones avanzadas como triggers, vistas e índices. Además, soporta conexiones externas mediante bibliotecas y drivers como libpq y ODBC, lo cual facilita su integración con otros lenguajes de programación y plataformas.

Dentro del lenguaje SQL, las operaciones más utilizadas para la manipulación de datos corresponden al conjunto CRUD (Create, Read, Update y Delete), las cuales permiten gestionar la información almacenada en una base de datos:

- **INSERT**: Inserta uno o varios registros nuevos dentro de una tabla.
- **SELECT**: Recupera y consulta la información almacenada según los criterios especificados.
- **UPDATE**: Modifica los valores de uno o varios registros existentes.
- **DELETE**: Elimina registros de una tabla de forma permanente cuando cumplen una condición determinada.

Estas instrucciones constituyen la base para la administración y mantenimiento de la información dentro de una base de datos relacional.

### GNU Octave

GNU Octave es un entorno de programación de alto nivel, principalmente orientado a cálculos numéricos. Su sintaxis es compatible con MATLAB, lo cual lo hace ideal para tareas de simulación, análisis de datos, procesamiento de señales, entre otras aplicaciones de ingeniería y ciencia.

El paquete `database` dentro de Octave permite realizar conexiones con bases de datos relacionales utilizando drivers como libpq o JDBC. Antes de utilizar sus funciones es necesario cargar el paquete mediante el comando:

```
pkg load database
```

Una vez cargado el paquete, se pueden utilizar funciones como:

- `pq_connect()`: Establece la conexión con la base de datos, especificando parámetros como host, puerto, usuario, contraseña y nombre de la base de datos.
- `pq_exec_params()`: Ejecuta sentencias SQL, permitiendo realizar consultas y operaciones como `INSERT`, `SELECT`, `UPDATE` y `DELETE`.
- `pq_close()`: Finaliza la conexión con la base de datos de forma segura.

Este tipo de conexión es útil para realizar consultas directamente desde un entorno de análisis numérico sin necesidad de exportar datos manualmente, promoviendo la automatización de flujos de trabajo.

* email:2996393200101@ingenieria.usac.edu.gt

### Python y la biblioteca psycopg2

Python es un lenguaje de programación de alto nivel ampliamente utilizado para el desarrollo de aplicaciones, automatización de procesos, análisis de datos y desarrollo científico. Gracias a su amplia variedad de bibliotecas, permite interactuar fácilmente con sistemas gestores de bases de datos como PostgreSQL.

La biblioteca `psycopg2` es uno de los conectores más utilizados para establecer comunicación entre Python y PostgreSQL. Esta biblioteca permite crear conexiones con la base de datos, ejecutar instrucciones SQL y administrar transacciones mediante métodos como `commit()`, garantizando que los cambios realizados sean almacenados de manera permanente.

A través de `psycopg2` es posible implementar las operaciones CRUD, ejecutando sentencias `INSERT`, `SELECT`, `UPDATE` y `DELETE`, además de recuperar los resultados de las consultas utilizando funciones como `fetchall()`, facilitando así el procesamiento y visualización de la información desde programas escritos en Python.

## III. ALGORITMO

- Inicio
- Instalar el paquete database en Octave.
- Cargar el paquete database.
- Establecer la conexión con la base de datos PostgreSQL.
- Verificar si la conexión fue exitosa.
- Si la conexión es correcta:

  - Insertar un registro en la tabla redes.
  - Consultar todos los registros de la tabla.
  - Mostrar los datos obtenidos.

- Si la conexión falla:

  - Mostrar un mensaje de error.

- Finalizar el programa.

## IV. DIAGRAMA DE FLUJO

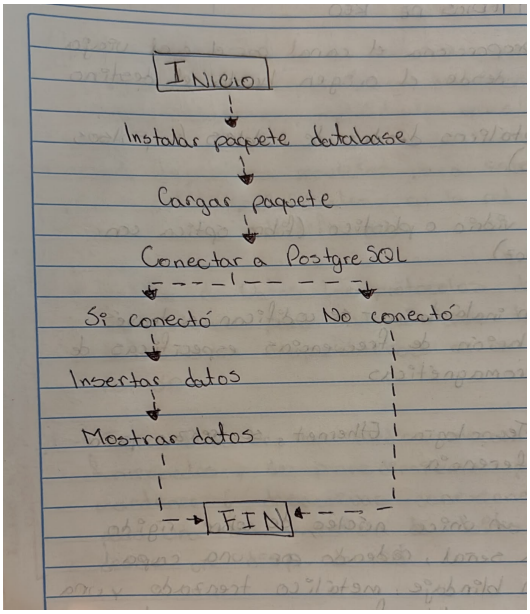



Figura 1: Diagrama de Flujo

## V. RESULTADOS

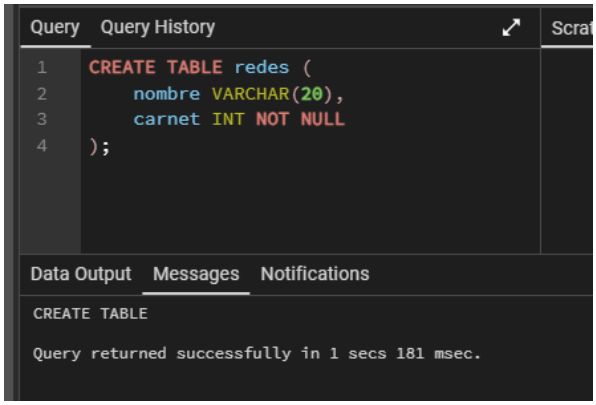



Figura 2: Creacion de Tabla

```
Query  Query History

-- INSERT: Agregar un nuevo registro
INSERT INTO redes (nombre, carnet)
VALUES ('Ana Morales', 202056789);

-- SELECT Mostrar todos los registros
SELECT * FROM redes;

-- UPDATE Modificar el nombre del registro
UPDATE redes
SET nombre = 'Ana Sofía Morales'
WHERE carnet = 202056789;

-- SELECT Verificar el cambio realizado
SELECT * FROM redes;

-- DELETE Eliminar el registro
DELETE FROM redes
WHERE carnet = 202056789;

-- SELECT Verificar que el registro fue eliminado
SELECT * FROM redes;
```

Figura 3: Script completo en pgAdmin

```
Tarea2.py U
Tarea2.py > ...
# INSERT
cursor.execute("""
INSERT INTO redes (nombre, carnet)
VALUES ('Carlos López', 202045678);
""")
conn.commit()

print("=== Después del INSERT ===")
cursor.execute("SELECT * FROM redes;")
for fila in cursor.fetchall():
    print(fila)

# UPDATE
cursor.execute("""
UPDATE redes
SET nombre = 'Carlos Alberto López'
WHERE carnet = 202045678;
""")
conn.commit()

print("\n=== Después del UPDATE ===")
cursor.execute("SELECT * FROM redes;")
for fila in cursor.fetchall():
    print(fila)

# DELETE
cursor.execute("""
DELETE FROM redes
WHERE carnet = 202045678;
""")
conn.commit()
```

Figura 4: Código completo en python

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\mafee\Documents\USAC\PROYECTOS\Tarea #2 - Postgres y Octave> & C:\Users\mafee\AppData\Local\Programs\Python\Python313\python.exe "d:/mafee/Documents/USAC/PROYECTOS/Tarea #2 - Postgres y Octave/Tarea2.py"
=== Después del INSERT ===
('Fernanda Alvarez', 20201537)
('MJuan Perez', 202012345)
('Carlos López', 202045678)

=== Después del UPDATE ===
('Fernanda Alvarez', 20201537)
('MJuan Perez', 202012345)
('Carlos Alberto López', 202045678)

=== Después del DELETE ===
('Fernanda Alvarez', 20201537)
('MJuan Perez', 202012345)
PS D:\mafee\Documents\USAC\PROYECTOS\Tarea #2 - Postgres y Octave>
```

Figura 5: Registro y comprobación de datos en la Tabla desde Python

```
Editor
File  Edit  View  Debug  Run  Help
funcion.m  Tarea2.m
pkg load database

conn = pq_connect(setdbopts(...
'dbname','postgres',...
'host','localhost',...
'port','5432',...
'user','postgres',...
'password','mafer123'));

% INSERT
pq_exec_params(conn,"INSERT INTO redes VALUES ('Maria',201854321);")

% SELECT
N = pq_exec_params(conn,"SELECT * FROM redes")

% UPDATE
pq_exec_params(conn,"UPDATE redes SET nombre='Maria Fernanda' WHERE

% SELECT
N = pq_exec_params(conn,"SELECT * FROM redes")

% DELETE
pq_exec_params(conn,"DELETE FROM redes WHERE carnet=201854321;");

% SELECT
line: 9  col: 1  encoding: UTF-8  eol: CRLF
Editor  Command Window  Documentation  Variable Editor
Profiler
```

Figura 6: Código completo en Octave

```
Command Window
conn = <PGconn object>
>> N = pq_exec_params(conn, ...
"INSERT INTO redes VALUES ('Juan Perez',202012345);")
N = 1
>> N = pq_exec_params(conn,"SELECT * FROM redes;")
N =

  scalar structure containing the fields:

    data =
    {
      [1,1] = Fernanda Alvarez
      [2,1] = MJuan Perez
      [1,2] = 20201537
      [2,2] = 202012345
    }

    columns =
    {
      [1,1] = nombre
      [1,2] = carnet
    }

    types =

      1x2 struct array containing the fields:

        name
        is_array
        is_composite
        is_enum
>>      elements
```

Figura 7: Registro y comprobación de datos en la Tabla desde Octave

## VI. CONCLUSIONES

Conectar Octave con PostgreSQL permite ampliar el alcance del análisis de datos, facilitando la inserción, actualización y visualización de información en tiempo real. Esta integración es útil en aplicaciones que requieren tratamiento numérico junto con almacenamiento estructurado. La práctica también resalta la importancia de una buena instalación y configuración de controladores.

## VII. ANEXOS

```
%Estructura de control de flujo
x = 1
y = 0
z = -5

if (x > y & z < 0)
  'X es mayor a Y y Z es menor a 0'
elseif (x == y | z < 0)
  'X y Y son iguales'
else
  'Y es mayor a X'
endif

while(z < y)
'Valor de z'
z
++z
endwhile

fib = ones(1,10)
for i = 3:10
  fib(i) = fib (i-1) + fib(i-2);
endfor

fib
```

Figura 8: Código video: estructuras de control de flujo

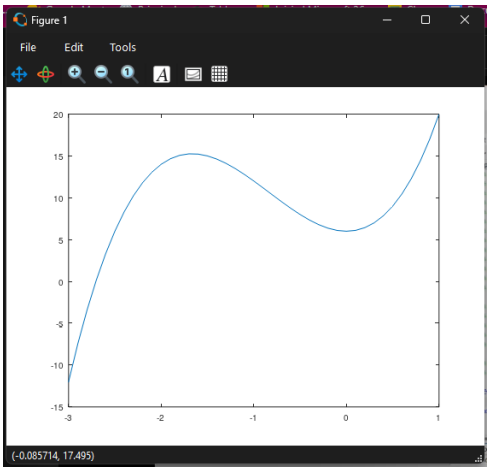



Figura 10: Gráfica de: $4x^3 + 10x^2 + 6$

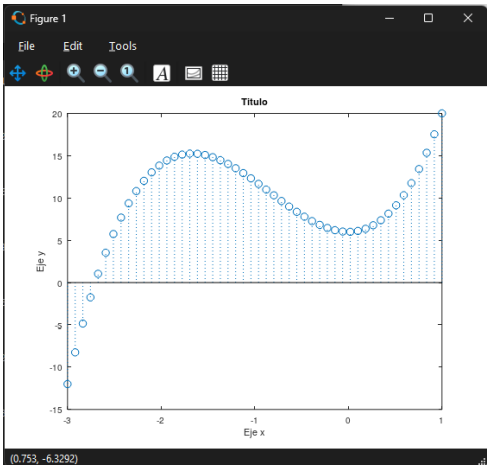



Figura 11: Gráfica de: $4x^3 + 10x^2 + 6$ discreta

```
%FUNCIONES
[x,b] = hipotenusa(2,2)

%GRAFICAS
x = [-3:0.1:1]; %Rango,saltos,rango
x = linspace(-3,1,50) %De donde, a donde, cuantos puntos
plot(x,funcion(x), 'LineStyle',':');
title('Titulo')
ylabel('Eje y')
xlabel('Eje x')

x = [-3:0.1:1]; %Rango,saltos,rango
x = linspace(-3,1,50) %De donde, a donde, cuantos puntos
stem(x,funcion(x), 'LineStyle',':');
title('Titulo')
ylabel('Eje y')
xlabel('Eje x')
legend('Funcion')

%FUNCION SENO Y COSENO
x = [0:0.1:4*pi];
y1 = sin(x);
y2 = cos(x);
hold on;
p1 = plot(x,y1);
p2 = plot(x,y2);
```

Figura 9: Código video: Funciones y Gráficas

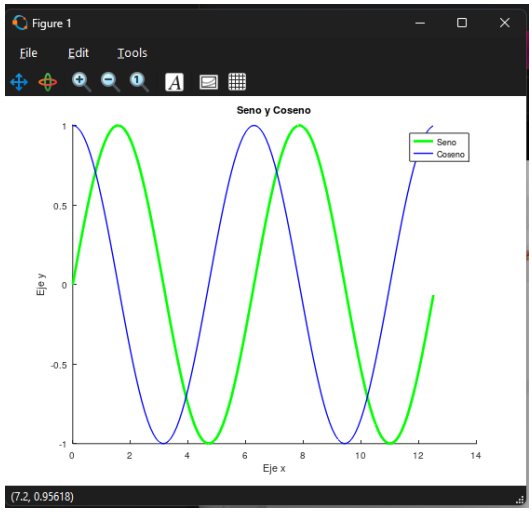



Figura 12: Gráfica de la función Seno y Coseno